

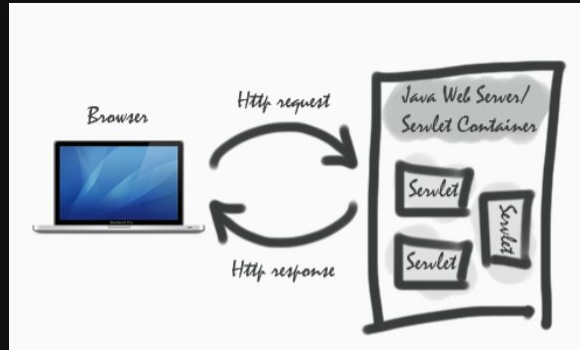
Java Web Programming

Created by <Academia de Código_>

Web Application

Collection of web components that work together to provide a specific functionality on the Web

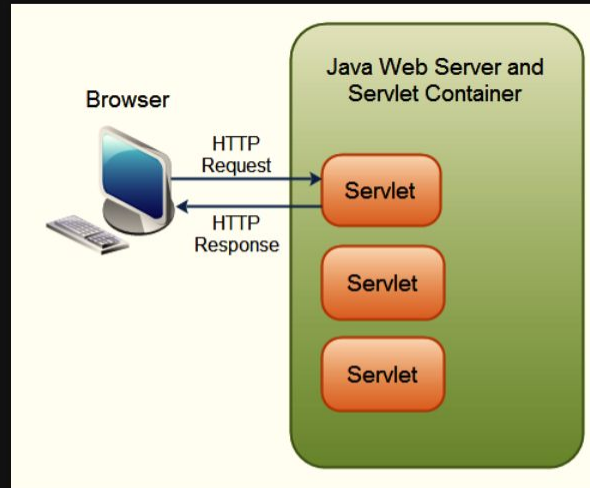
In the JEE specification, a web component is either a Servlet, JavaServer Faces or a Java Server Page (JSP) page



Web applications are modeled after the request-response stateless nature of the HTTP protocol

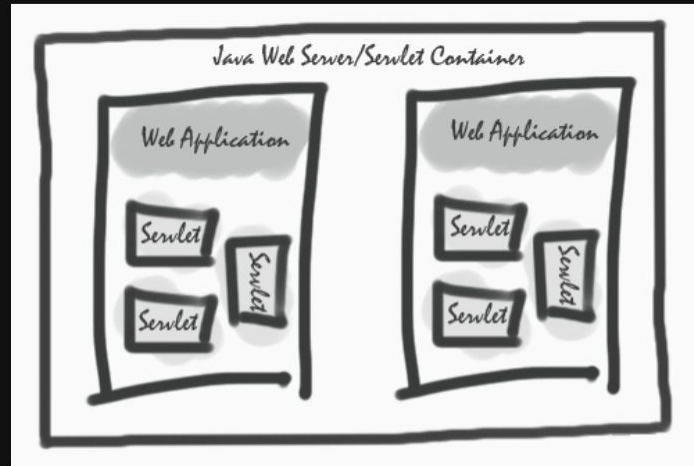
Servlet

Core classes in web applications that either perform the work of responding to requests or delegate that work to some other part of the application



Servlet Container

The web application and its constituent components are managed and executed inside the web container, also called a servlet container.



Java Server Pages

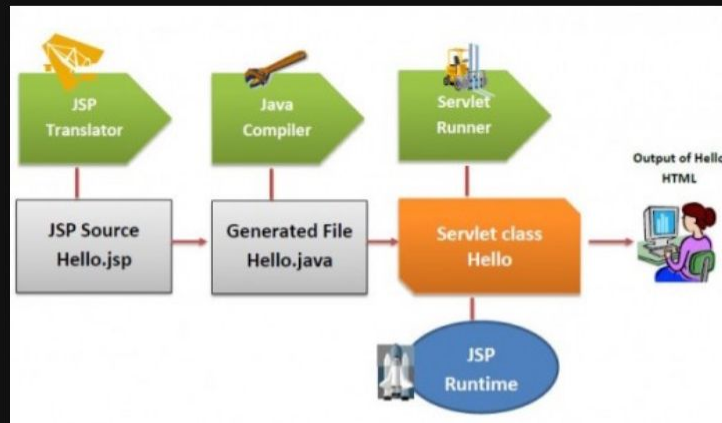
JSP is an extension of the servlet technology allowing java programming code to be embedded in HTML pages

The Java code is executed on the server and compiled with the JSP to produce a plain HTML page



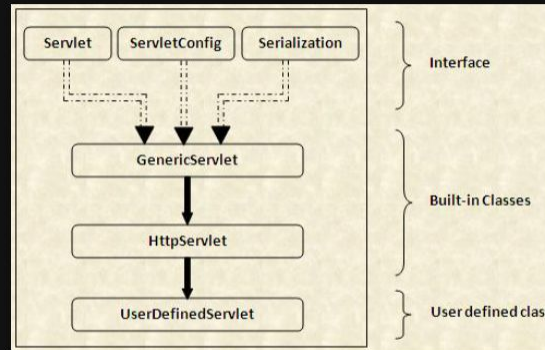
Translation Process

JSP files are first compiled into Servlets before they can be used



Servlet Class Hierarchy

- GenericServlet provides protocol-independent basic servlet functionality
- HttpServlet inspects the incoming HTTP request and invokes the appropriate method for the request type
- ServletConfig is used to pass initialization parameters to the servlet and access to the ServletContext



Servlet API

The `javax.servlet.Servlet` interface defines lifecycle methods to initialize a servlet, to service requests and to remove a servlet from the server

```
service(ServletRequest, ServletResponse)
init(ServletConfig)
destroy()

getServletConfig()
getServletInfo()
```

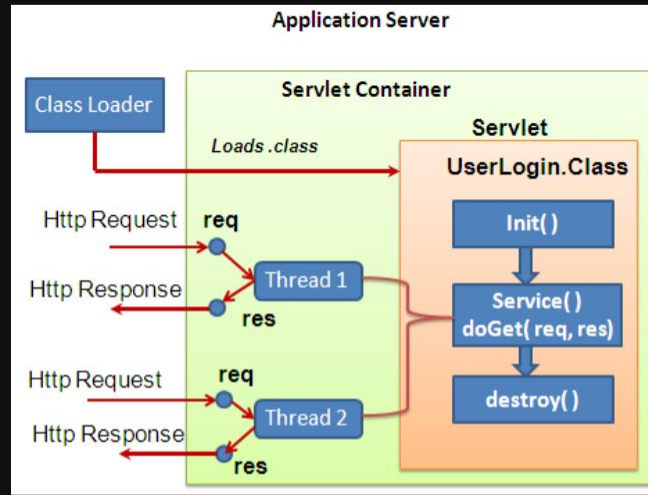
Life Cycle method

Non Life Cycle method

The interface also provides the `getServletConfig` method, which the servlet can use to get any startup information and the `getServletInfo` method, which allows the servlet to return basic information about itself

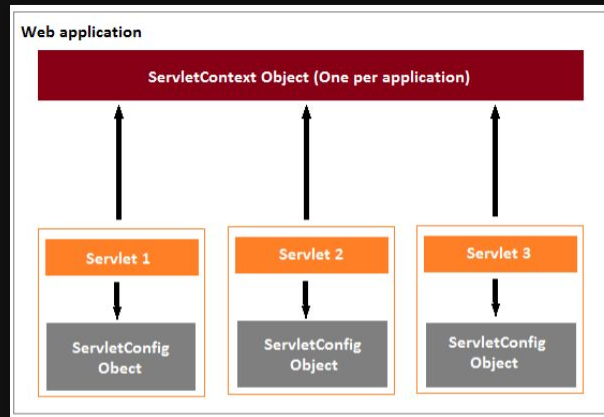
Servlet Lifecycle

The Servlet life-cycle methods are invoked by the container at appropriate instants in a servlet's life:



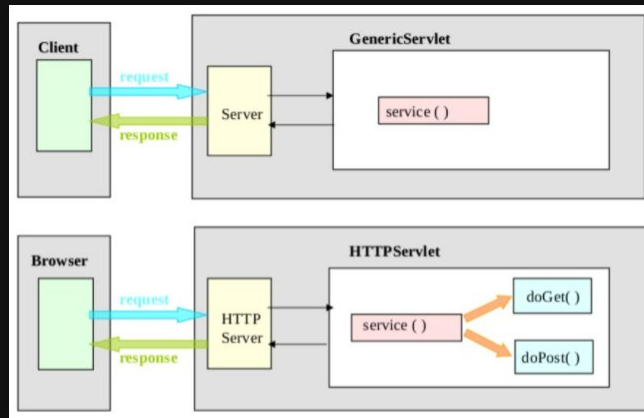
ServletContext

Can be used to access application parameters (request, initialization and context), set globally accessible attributes and to obtain container information



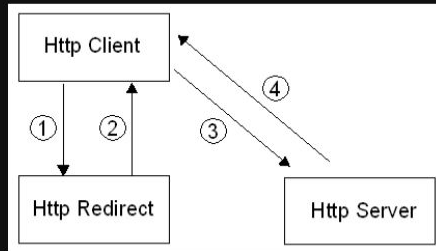
HttpServlet

`javax.servlet.http.HttpServlet` extends `GenericServlet` and implements the service method to accept only HTTP requests, providing empty implementations for methods corresponding to each HTTP verb type



Redirection

In a web application the request flow can be redirected to a completely different URL

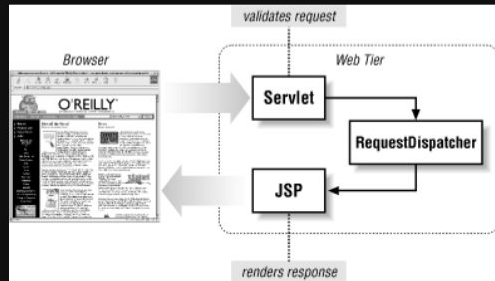


The redirect is done by the browser and can be triggered by calling `sendRedirect()` on the response object.

```
public class LoginServlet extends HttpServlet {  
  
    @Override  
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {  
  
        resp.sendRedirect("http://javabank.com/login");  
    }  
}
```

Dispatching

Requests can be dispatched to other application components such as Servlets or JSP pages

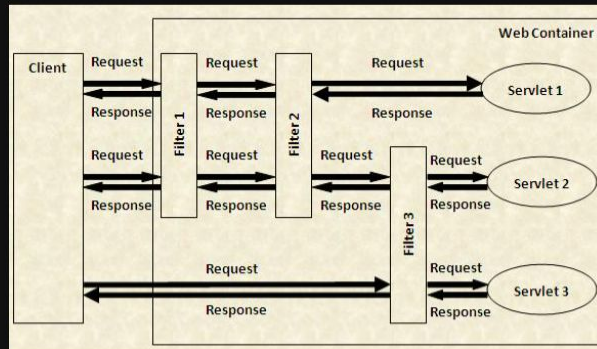


A request dispatch is different from a redirect, the work is done on the server side and not by the browser

```
public class LoginServlet extends HttpServlet {  
    @Override  
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {  
        RequestDispatcher dispatcher = getServletContext().getRequestDispatcher("customer.jsp");  
        dispatcher.forward(req, resp);  
    }  
}
```

Filters

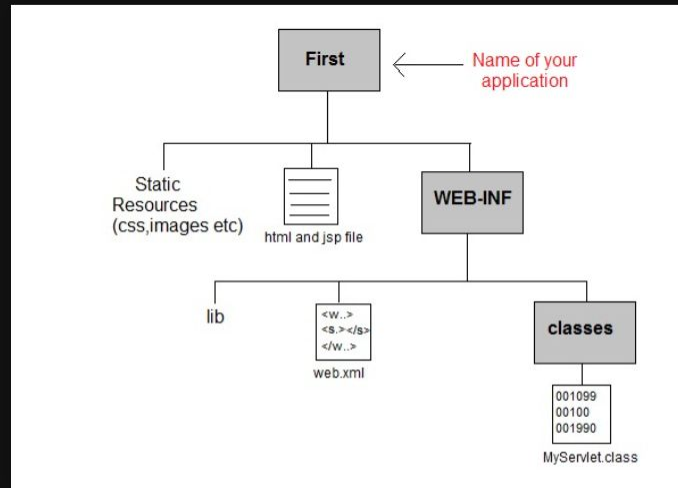
Filters are application components organized in a chain that can intercept, inspect and modify requests and responses



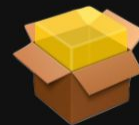
Filters implement the [javax.servlet.Filter](#) interface, are configured on the deployment descriptor and are typically used for authentication, caching, encryption, logging, auditing, etc.

Web Applications Directory Structure

Web Applications follow a directory structure convention, which contains the deployment descriptor, classes and other application resources



Web Application Archive



A WAR file is usually used to distribute a collection of JavaServer Pages, Java Servlets, Java classes, XML files, tag libraries, static Web pages (HTML and related files) and other resources that together constitute a Web application

In essence, a WAR file is really a JAR file with some additional web specific metadata

The Deployment Descriptor

At the heart of all web applications is a deployment descriptor that is an XML file named web.xml and contains configuration information for the entire web application.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0"
  xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
  <servlet>
    <servlet-name>hello</servlet-name>
    <servlet-class>MyServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>hello</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>
</web-app>
```

First line of any xml document

root tag of web.xml file. All other tag come inside it

this tag maps internal name to fully qualified class name

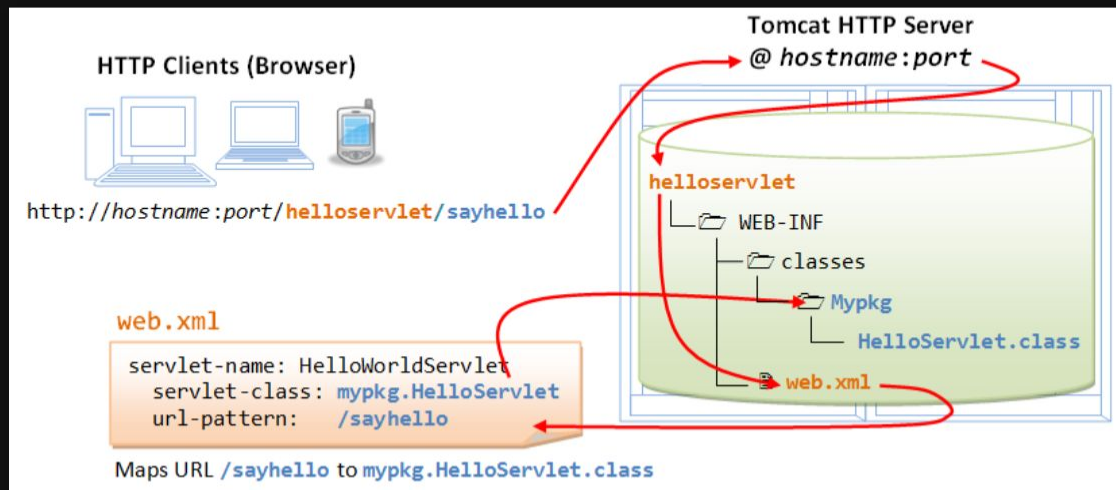
Give a internal name to your servlet

servlet class that you have created

this tag maps internal name to public URL name

URL name. This is what the user will see to get to the servlet.

Servlet URL Mapping



- a path matching pattern is an url pattern which starts from / and ends with *
- if a servlet is mapped with url pattern '/' then it is called the default servlet
- if a servlet is mapped with empty string url pattern then it is mapped the root of the web application

Apache Tomcat

Java Based Web Application Container used to deploy Java applications that are built using the Servlet and JSP APIs



Maven Tomcat Plugin

Provides goals to deploy and manipulate WAR projects within the Apache Tomcat servlet container

Once a WAR project has been deployed on Tomcat, it is possible to:

- Redeploy a WAR project
- Undeploy a WAR project
- Start a WAR project
- Stop a WAR project
- List session statistics

Maven Tomcat Configuration

```
<plugin>
  <groupId>org.apache.tomcat.maven</groupId>
  <artifactId>tomcat7-maven-plugin</artifactId>
  <version>2.2</version>
  <configuration>
    <url>http://localhost:8080/manager/text</url>
    <!-- server defined in ~/.m2/settings.xml -->
    <server>tomcat</server>
    <path>/hello</path>
  </configuration>
</plugin>
```

{ Hands On }

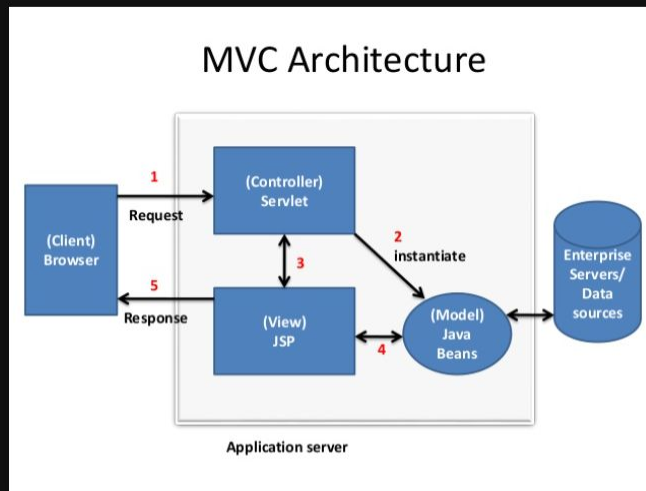
Install and Configure Tomcat Server

{ Hands On }

Hello Servlet

MVC with Servlets and JSPs

- Model is represented by a POJO object that carries data
- View layer will present the data using a JSP page
- Controller logic will be implemented by the Servlet



Request Dispatcher

The View can be presented by forwarding the request to the JSP using a RequestDispatcher

The RequestDispatcher object receives requests from the client and sends them to any resource on the server, such as a servlet, HTML file or JSP file

```
// Create request dispatcher wrappers around the views
RequestDispatcher page1Dispatcher = getServletContext().getRequestDispatcher("page1.jsp");
RequestDispatcher page2Dispatcher = getServletContext().getRequestDispatcher("page2.jsp");

// include the page1 processing by calling the appropriate jsp page code
page1Dispatcher.include(request, response);

// transfer the control to page2 jsp
page2Dispatcher.forward(request, response);
```

Populating the View

Before handling the processing of the request to the view, it needs to be populated with model data

```
User user = new User();
user.setName("Catarina Campino");
user.setEmail("catarina.campino@academiadecodigo.org");

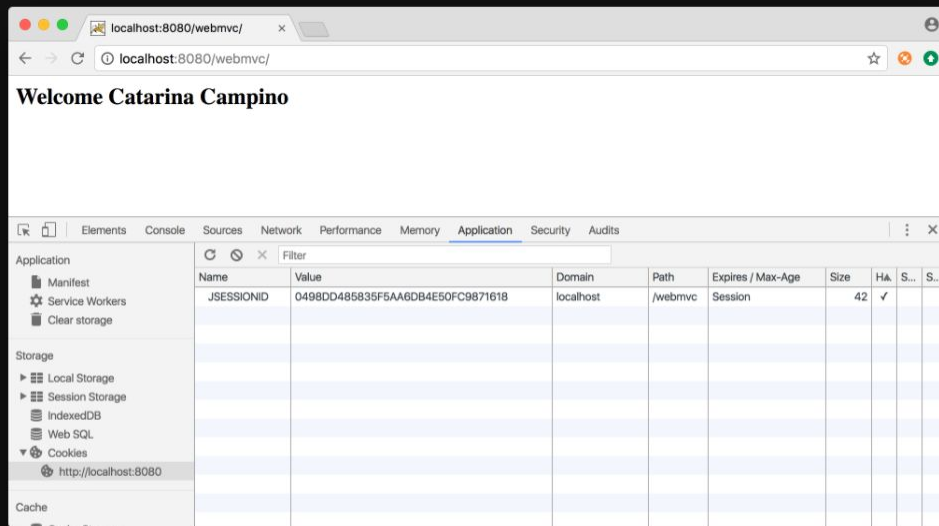
// Use global Servlet context to pass user object
getServletContext().setAttribute("user", user);

// Use session context to pass the user object
request.getSession().setAttribute("user", user);

// Use request context to pass the user object
request.setAttribute("user", user);
```

Tomcat Sessions

Session state is maintained in Tomcat using a JSESSIONID cookie, which is created the first time request.getSession()



Expression Language

Allows dynamic access to model data stored as attributes in the request, session or application scope

```
<html>
  <head></head>
  <body>
    <h2>Welcome ${user.name}</h2>
    <p>Email: ${user.email}</p>
  </body>
</html>
```

Metadata

The apache maven-archetype-webapp can be utilized to create a basic web application structure

Make sure the maven compiler and tomcat plugins are installed and configured and the servlet-api dependency is added

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>3.1.0</version>
  <scope>provided</scope>
</dependency>
```

The provided scope means the dependency will be provided at runtime by the container

{ Exercise }

Web MVC Customer Details

Web Forms

Web Forms

Web forms allow collecting data from the user.

A **form** is made up of **form controls**.

There are a number of available **form controls** to gather any type of data from the user.

```
<form>
  <input name="username" type="text" />
  <input name="password" type="password" />

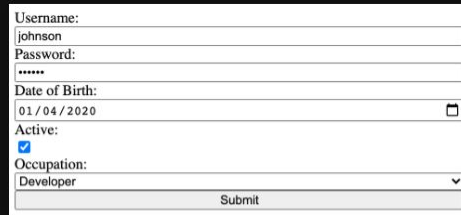
  <input type="submit" value="Submit" />
</form>
```

The <input> element

The majority of available form controls are <input> elements.

<input> elements' look and behaviour considerably change depending on the value of the type attribute.

```
<form class="flex columns">
  <label for="username">Username:</label>
  <input name="username" type="text" />
  <label for="password">Password:</label>
  <input name="password" type="password" />
  <label for="birthdate">Date of Birth:</label>
  <input name="birthdate" type="date" />
  <label for="active">Active:</label>
  <input name="active" type="checkbox" />
  <label for="occupation">Occupation:</label>
  <select name="occupation">
    <option value="developer">Developer</option>
    <option value="designer">Designer</option>
  </select>
  <input type="submit" value="Submit" />
</form>
```



Username:
johnson

Password:

Date of Birth:
01 / 04 / 2020

Active:
☒

Occupation:
Developer

Submit

<input type="submit"> renders a button that, **by default**, submits all the data in its containing form through an HTTP request.

{ Hands On }

Form Controls

Form element reference

Input element reference

Form Submission

By default, form is submitted through a GET request to the current document's url. The data is included in the query parameters.

`<form>` element's `action` and `method` attributes allow customizing this behaviour.

```
<form action="http://localhost:8000/form-handler" method="POST">
  <!-- form controls.. -->
  <input type="submit" value="Submit" />
</form>
```

In this example, form data will be sent through a **POST** request to `http://localhost:8000/form-handler`

Handling Form Submission

Form data sent through a POST request payload has the following format:

`name=value&name=value`

We need to parse it with server side code for further processing!

Handling Form Submission

Form data sent through a POST request is included in the payload.

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp) {

    String name = req.getParameter("username");

}
```

{ Hands-On }

Form Handling with Java Servlets

Data Validation

Validation ensures data is in the desired format.

It's **extremely important** to **validate** data that comes from **external sources**, like web forms.

Validation not only ensures our application is dealing with the right type of data, but also protects the application against malicious clients.

Data Validation

In web applications, data validation may be done **client** or **server** side.

Client-side validation provides better user experience, but may be bypassed by malicious users.

Client-side Form Validation

Client-side form validation may be applied directly on HTML form control elements.

Built-in form validation is very performant, requires very little coding, and covers most validation needs.

```
<!-- built-in form validation -->
<input type="text" name="username" required minlength="4" maxlength="12"/>
<input type="number" name="amount" min="1" max="10"/>
<input type="email" name="email" />
```

Built-in Form Validation

There are set of form control element attributes that may be used to validate their value

The form won't be submitted to the server if there's one or more form fields failing validation.

- **required**: the form field must have a value.
- **minlength** and **maxlength**: specifies minimum and maximum length of a text field value.
- **min** and **max**: specifies the minimum and maximum values for numeric values
- **type**: specifies the type of data that needs to be filled in. May include extra validation (e.g: email, tel, ...etc)
- **pattern**: defines a pattern that the entered data needs to follow, using **Regular Expressions**

Validation with Style

CSS pseudo-class selectors are available for styling form controls depending on their validation status.

```
input:required {  
  outline: 1px solid red;  
}  
  
input:invalid {  
  background-color: red;  
}  
  
input:valid {  
  background-color: green;  
}  
  
#email-input:invalid {  
  background-color: tomato;  
  outline: 2px solid red;  
}
```

{ Exercise }

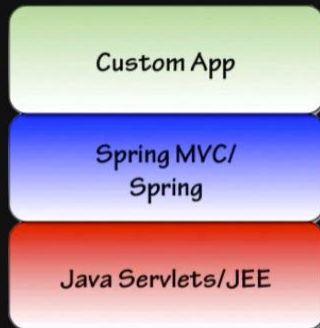
Add validation to your Form HTML elements



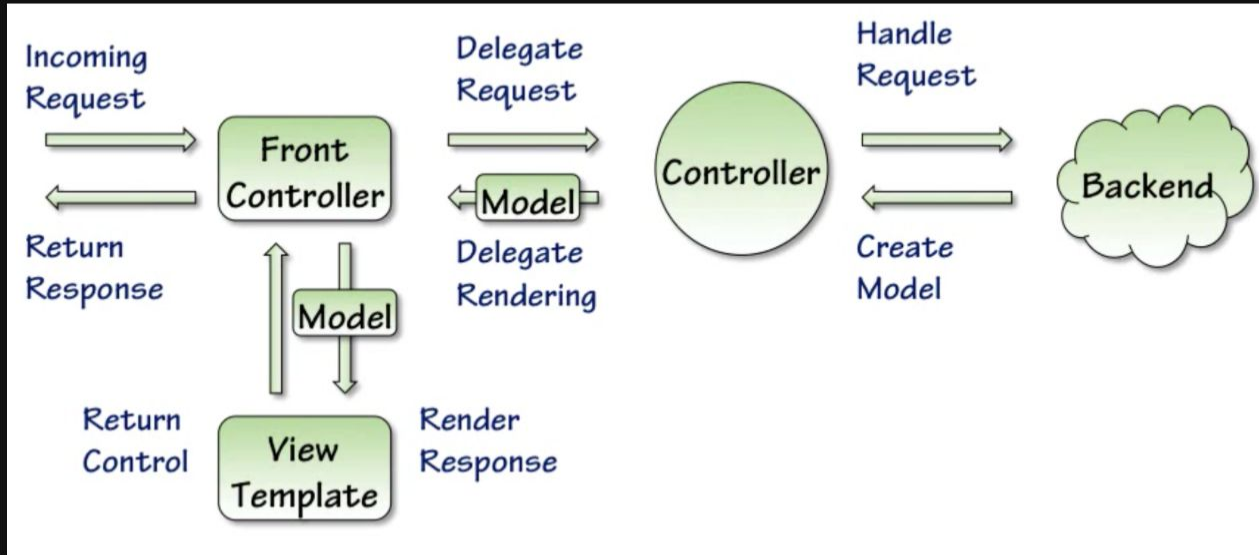
Spring MVC

Higher level abstraction built on top of Java Servlets and centered around the Model View Controller design pattern

Speeds up development time by providing general solutions to commonly well-known problems



Architecture Overview

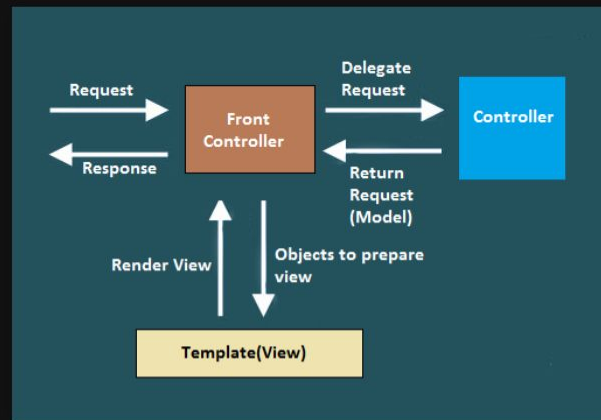
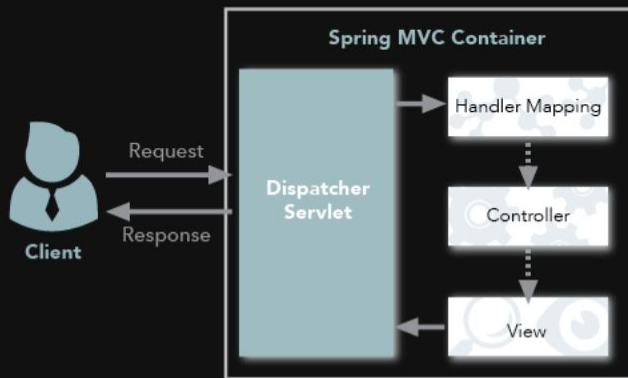


Spring uses the Front Controller design pattern

Servlets are replaced with controllers

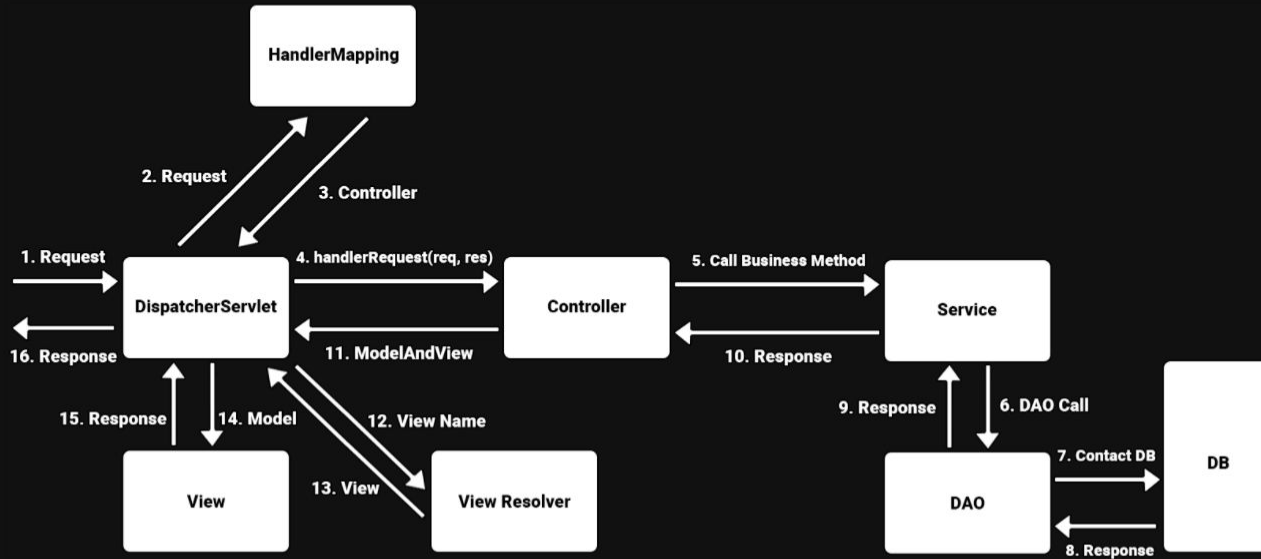
Dispatcher Servlet

- Maps and delegates requests to controllers
- Resolves the appropriate view



DispatcherServlet and HandlerMapping beans are provided by the framework

End-to-End Request Flow



Spring Controller

Spring will register controllers and their mappings by scanning your code for the @Controller and @RequestMapping annotations

```
// Let Spring find the controller
@Controller
public class HelloController {

    // Map the URL/Verb to the method
    @RequestMapping(method = RequestMethod.GET, value = "/hello")
    public String sayHello(Model model) {

        // Add an attribute to the request
        model.addAttribute("greeting", "Hello World");

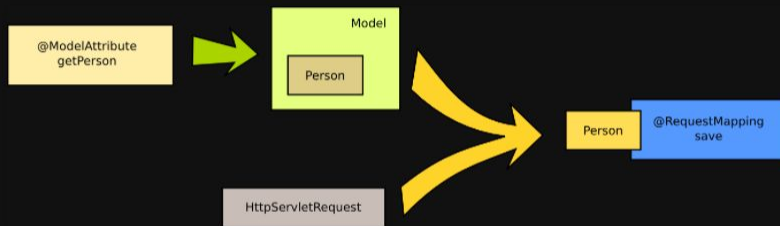
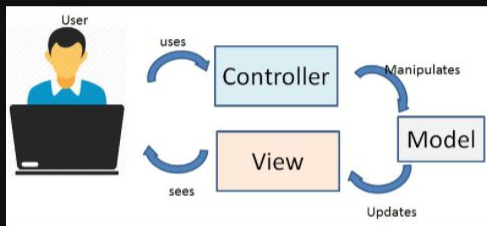
        // Return the view name
        return "hello";
    }
}
```

Model data can be conveniently added to the request through the usage of an org.springframework.ui.Model object as argument

Spring Model

The model is a Map interface defining a holder for model attributes and allowing for the complete abstraction of the view technology

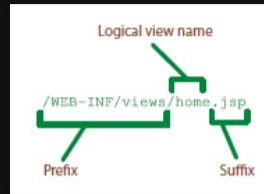
- Data can be added to the Spring model inside the controller using the Model or ModelAndView classes
- Data can be added to the Spring model by annotating methods with ModelAttribute
- Data can be retrieved from the Spring model by annotating controller method arguments with ModelAttribute
- Spring model attributes are copied to the request scope before rendering the view



View Resolver

The ViewResolver interface provides a mapping between view names and the actual view objects

InternalResourceViewResolver is a convenience ViewResolver implementation for using JSP pages as view templates



The view resolver surrounds the view name with a prefix and a suffix providing the dispatcher servlet with the file path to the template

View Template

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
  <head>
    <title>Hello Spring MVC</title>
  </head>
  <body>
    <h1>${greeting}</h1>
  </body>
</html>
```

Model data is accessed using the JSP Expression Language

Spring Configuration

```
<beans>

  <!-- Use spring servlet for all requests, including static resources -->
  <mvc:default-servlet-handler/>

  <!-- look for spring annotations inside the controller package -->
  <mvc:annotation-driven/>
  <context:component-scan base-package="org.academiadecodigo.bootcamp.controller"/>

  <!-- configure the view resolver -->
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/templates/" />
    <property name="suffix" value=".jsp" />
  </bean>

</beans>
```

Bootstrapping Spring

The Spring application context is loaded from the DispatcherServlet, which is configured in the project deployment descriptor:

```
<servlet>
  <servlet-name>dispatcherServlet</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>WEB-INF/spring/spring-config.xml</param-value>
  </init-param>
</servlet>

<servlet-mapping>
  <servlet-name>dispatcherServlet</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

Project Dependencies

```
<dependencies>

  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>4.3.11.RELEASE</version>
  </dependency>

  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
    <scope>provided</scope>
  </dependency>

</dependencies>
```

Add these to maven-archetype-webapp project

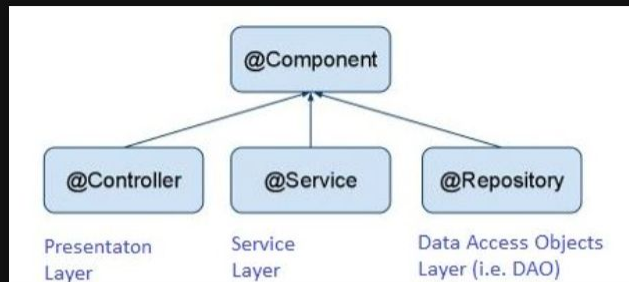
{ Exercise }

Spring MVC Customer Details

Auto Wiring

Spring auto wiring enables object dependencies to be injected implicitly without the need for explicit configuration

Beans can be registered on the Spring container by using one of the available annotations:



A Spring managed bean can be injected into a property simply by using the @Autowired annotation

Spring Profiles

Beans can be registered on the container only when the appropriate profiles are active:

```
@Service
@Profile("jpa")
public class CustomerServiceImpl implements CustomerService {}

@Service
@Profile("memory")
public class MockCustomerService implements CustomerService {}

@Controller
@RequestMapping("/customer")
public class CustomerController {

    private CustomerService customerService;

    @Autowired
    public void setCustomerService(CustomerService customerService) {}
}
```

Active profile selection can be done through servlet context parameter:

```
<context-param>
  <param-name>spring.profiles.active</param-name>
  <param-value>jpa</param-value>
</context-param>
```

Template Engines

Spring allows the usage of different view technologies, from the well established JSP technology to a variety of template engines



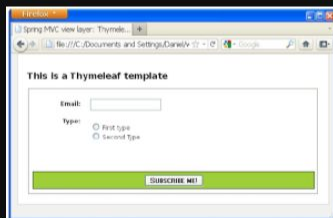
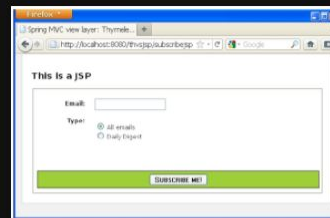
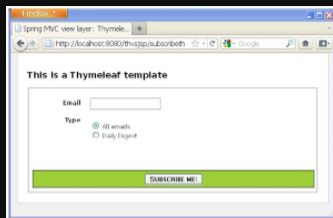
To render each view type, a ViewResolver bean corresponding to the template engine technology needs to be defined

Thymeleaf



Modern Java template engine which can process HTML, XML, text, JavaScript or CSS files

Supports Natural Templating, allowing the template to be correctly displayed in the browser and work as a static prototype



Model Attributes

For simple model attributes, the **th:text** tag attribute can be used to display the value of model attributes:

```
<h2 th:text="${'Welcome ' + user.firstName + ' ' + user.lastName}">Welcome Bruno Ferreira</h2>
```

For iterating over a collection, the **th:each** tag attribute can be used:

```
<table>
  <tr th:each="student: ${students}">
    <td th:text="${student.id}" />
    <td th:text="${student.name}" />
  </tr>
</table>
```

Conditional Evaluation

The **th:if** attribute is used to display a section of the view if the condition is met

The **th:unless** attribute is used to display a section of the view if the condition is not met

```
<p>
  <span th:if="${student.gender} == 'M'" th:text="Male" />
  <span th:unless="${student.gender} == 'M'" th:text="Female" />
</p>
```

Conditional Evaluation

The **th:switch** and **th:case** attributes are used to display content conditionally:

```
<p th:switch="${student.gender}">
  <span th:case="'M'" th:text="Male" />
  <span th:case="'F'" th:text="Female" />
</p>
```

Thymeleaf Template

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <title>JavaBank</title>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
</head>
<body>
  <div>
    <h2>Customer List</h2>
    <table>
      <tr>
        <th>Id</th>
        <th>First Name</th>
        <th>Last Name</th>
        <th>Email</th>
        <th>Phone</th>
      </tr>
      <tr th:each="customer : ${customers}">
        <td th:text="${customer.id}"></td>
        <td th:text="${customer.firstName}"></td>
        <td th:text="${customer.lastName}"></td>
        <td th:text="${customer.email}"></td>
        <td th:text="${customer.phone}"></td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Spring Configuration

To integrate Thymeleaf with Spring, a new ViewResolver implementation bean needs to be provided:

```
<!-- Thymeleaf View Resolver, implements Spring ViewResolver interface -->
<bean id="viewResolver" class="org.thymeleaf.spring4.view.ThymeleafViewResolver">
  <property name="templateEngine" ref="templateEngine"/>
  <property name="characterEncoding" value="UTF-8"/>
  <property name="contentType" value="text/html; charset=UTF-8"/>
</bean>
```

The ThymeleafViewResolver bean requires two additional beans from the Thymeleaf project to work:

```
<!-- Thymeleaf Template Engine (Spring4-specific version) -->
<bean id="templateEngine" class="org.thymeleaf.spring4.SpringTemplateEngine">
  <property name="templateResolvers">
    <set>
      <ref bean="templateResolver"/>
    </set>
  </property>
</bean>

<!-- Thymeleaf template resolver -->
<bean id="templateResolver" class="org.thymeleaf.spring4.templateresolver.SpringResourceTemplateResolver">
  <property name="prefix" value="/templates/" />
  <property name="suffix" value=".html" />
  <property name="templateMode" value="HTML" />
  <property name="cacheable" value="false" />
  <property name="characterEncoding" value="UTF-8" />
</bean>
```

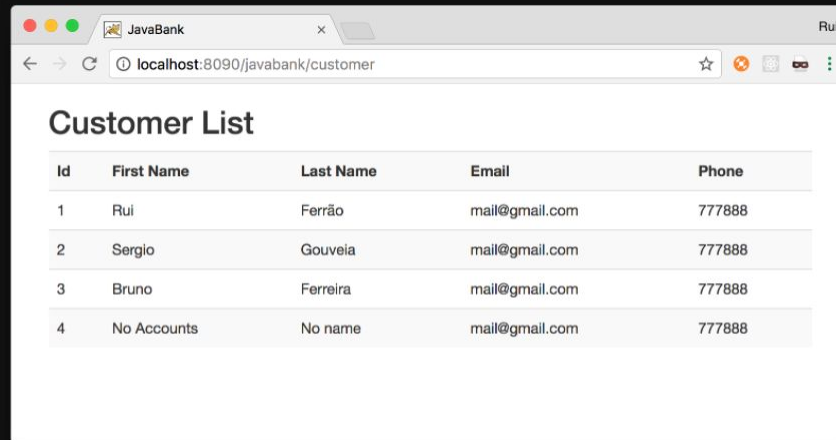
Metadata

The Spring Thymeleaf package needs to be added to the project metadata:

```
<dependency>  
  <groupId>org.thymeleaf</groupId>  
  <artifactId>thymeleaf-spring4</artifactId>  
  <version>3.0.8.RELEASE</version>  
</dependency>
```

{ Exercise }

JavaBank Customer list view using Spring MVC and Thymeleaf



The screenshot shows a web browser window with the title 'JavaBank' and the URL 'localhost:8090/javabank/customer'. The page displays a 'Customer List' table with the following data:

Id	First Name	Last Name	Email	Phone
1	Rui	Ferrão	mail@gmail.com	777888
2	Sergio	Gouveia	mail@gmail.com	777888
3	Bruno	Ferreira	mail@gmail.com	777888
4	No Accounts	No name	mail@gmail.com	777888

Request Parameters

The `RequestParam` annotation indicates that a method parameter should be bound to an URL request parameter

```
// Can serve URLs like http://www.someserver.org/someapp/hello?name=catarina
@RequestMapping(method = RequestMethod.GET, value = "/hello")
public String hello(Model model, @RequestParam("name") String name) {

    model.addAttribute("message", "Hello " + name);
    return "hello";
}
```

URL templates

URL-like string, containing one or more variable names. When these variables are substituted by values, the template becomes a URL

- URL template: `http://www.javabank.com/customer/{id}`
- URL `http://www.javabank.com/customer/2`

The @PathVariable annotation can be used on a controller method argument to bind it to the value of a URI template variable:

```
@RequestMapping(method = RequestMethod.GET, path = "/customer/{id}")
public String getCustomer(@PathVariable Integer id) {

    // User asked for a specific customer

}
```

Session Attributes

The `SessionAttributes` annotation is used on a controller to designate which model attributes should be stored in the session before rendering the next view

```
@Controller
@SessionAttributes("customer")
public class LoginController {

    // store accessing customer in the http session

}
```

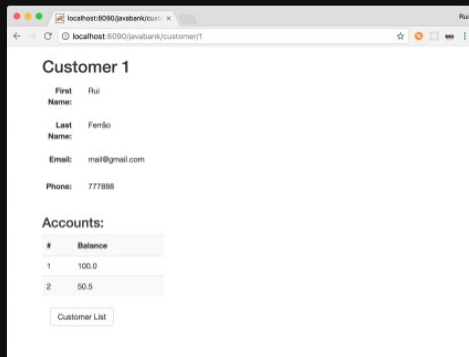
Spring Redirect

A Browser redirect can be triggered by returning from the controller the redirect: prefix followed by the URL:

```
public String deleteCustomer(@PathVariable Integer id) {  
    customerService.delete(id);  
    return "redirect:/customer";  
}
```

{ Exercise }

Add the Customer show and delete features to the JavaBank application



Customer 1

First Name: Rui

Last Name: Faria

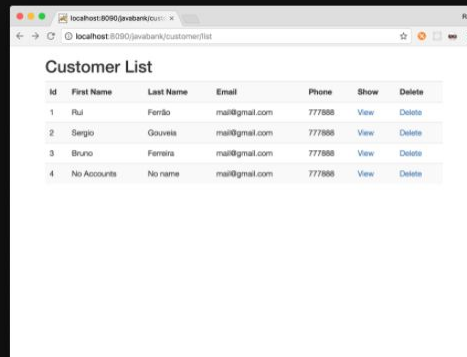
Email: mail@gmail.com

Phone: 777888

Accounts:

#	Balance
1	100.0
2	50.5

[Customer List](#)



Id	First Name	Last Name	Email	Phone	Show	Delete
1	Rui	Faria	mail@gmail.com	777888	View	Delete
2	Bergio	Gouveia	mail@gmail.com	777888	View	Delete
3	Bruno	Ferreira	mail@gmail.com	777888	View	Delete
4	No Accounts	No name	mail@gmail.com	777888	View	Delete

Spring Command Objects

Command objects (or form objects, form-backing objects) allow binding of form request parameters to bean properties using setters

- The **th:object** attribute inside a <form> tag tells thymeleaf which command object to bind to the form
- The **th:field** attribute attached to an html input field binds the specific field to the command object property

```
<form th:object="${customer}" th:action="@{/customer}" method="post">  
  <input type="text" th:field="*{firstName}"/>  
</form>
```

Values for **th:field** attributes must be selection expressions because they are binded for command objects and not model attributes

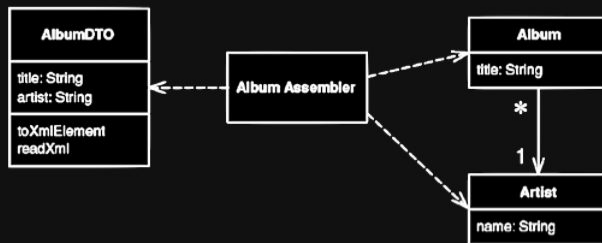
Thymeleaf Expressions

Thymeleaf attributes have their values set by using one of the following expression syntax:

- `${...}` Variable Expressions - Access to model data
- `*{...}` Selection Variable Expressions - Access properties of previously selected objects
- `@{...}` Link URL Expressions - Absolute or relative URLs
- `#{...}` Message Expressions - Access messages from resource bundles (eg: properties files with multiple langs)

DTO Pattern

Data Transfer Object is a design pattern used to transport information from a remote client to a server and vice-versa



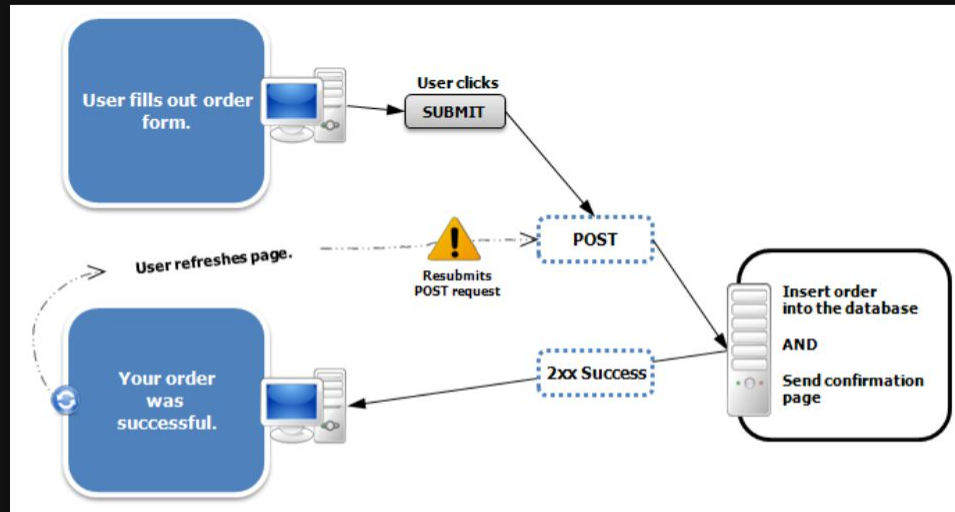
Using DTOs instead of models to transfer data to/from Spring backed forms we separate presentation logic from persistence logic

A DTO is just a collection of properties with getters and setters and should NOT have complex associations

A converter/assembler is required on the server to transfer data between DTOs and domain/model objects

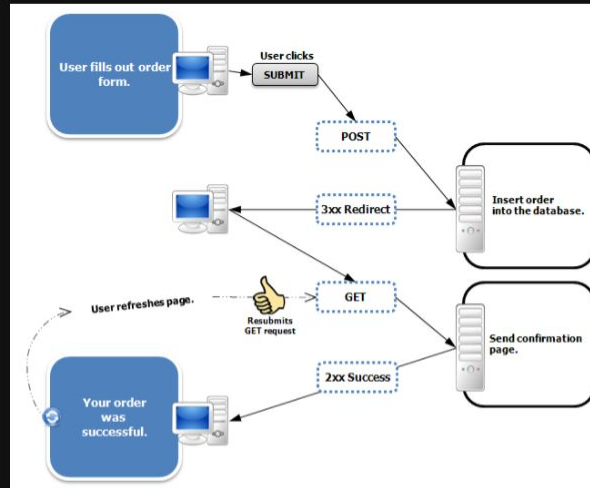
Duplicate Form Submission Problem

If the controller processing a form submission returns a success view, hitting the refresh browser button may trigger a new submission of the form with the same form data:



Post/Redirect/Get Pattern

PRG is a web development design pattern that prevents duplicate form submissions



PRG supports bookmarks and allows the browser refresh button to work in a predictable way

Spring Redirects

To implement PRG the controller handling the request needs to redirect the browser to another URL:

```
@Controller
@RequestMapping("/customer")
public class CustomerController {

    @RequestMapping(method = RequestMethod.GET, path =("/{id}")
    public String showCustomer(@PathVariable Integer id, Model model) {
        model.addAttribute("customer", customerService.get(id));
        return "customer/show";
    }

    @RequestMapping(method = RequestMethod.POST, value = "/add")
    public String addCustomer(@ModelAttribute Customer customer) {

        Customer savedCustomer = customerService.save(customer);

        // Instead of returning a rendered view to the browser,
        // a 302 redirect is sent to the browser, forcing showCustomer()
        // to execute after adding a new customer
        return "redirect:/customer/" + savedCustomer.getId();
    }
}
```

Flash Attributes

When redirecting a request, data stored in the original request is lost, making it impossible to the controller handling the redirected request to access the parameters contained in the original one

PRG solves multiple form submissions but prevents retrieving request parameters and attributes
Flash attributes provide a way for one request to store attributes intended for use in another request

```
@Controller
@RequestMapping("customer")
public class CustomerController {

    @RequestMapping(method = RequestMethod.GET, path = {"/list", "/", ""})
    public String listCustomers(Model model) {
        model.addAttribute("customers", customerService.list());
        return "customer/list";
    }

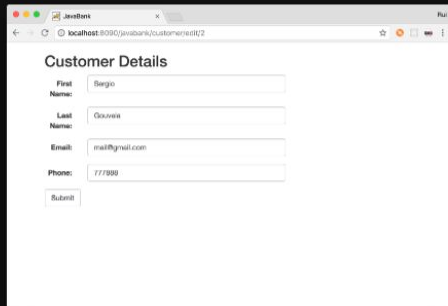
    @RequestMapping(method = RequestMethod.POST, path = {"/", ""})
    public String saveCustomer(@ModelAttribute CustomerDto customerDto, RedirectAttributes redirectAttributes) {

        Customer savedCustomer = customerService.save(customer);
        redirectAttributes.addFlashAttribute("lastAction", "Added customer successfully!");
        return "redirect:/customer/list";
    }
}
```

Flash attributes are saved temporarily before the redirect and removed immediately after

{ Exercise }

Add the Customer Add and Edit features to the JavaBank application



A screenshot of a web browser showing the 'Customer Details' form. The form has fields for First Name, Last Name, Email, and Phone, each with a 'Submit' button below it. The values entered are: First Name: Sergio, Last Name: Oliveira, Email: mail@gmail.com, and Phone: 777888.

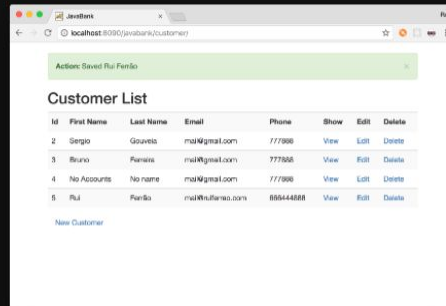
Customer Details

First Name:

Last Name:

Email:

Phone:



A screenshot of a web browser showing the 'Customer List' page. At the top, there is a green notification bar that says 'Action: Saved Rui Ferreira'. Below the notification is a table with columns: Id, First Name, Last Name, Email, Phone, Show, Edit, and Delete. The table contains five rows of customer data. At the bottom, there is a link 'New Customer'.

Action: Saved Rui Ferreira

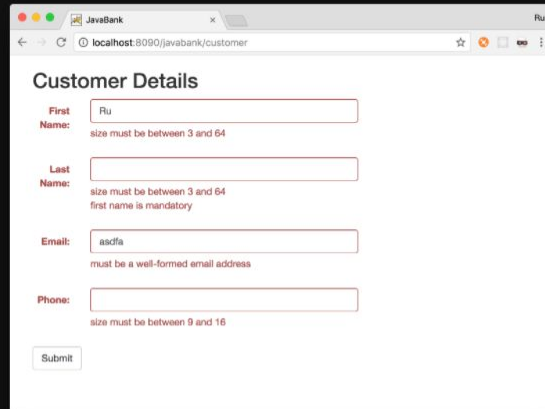
Customer List

Id	First Name	Last Name	Email	Phone	Show	Edit	Delete
2	Sergio	Oliveira	mail@gmail.com	777888	View	Edit	Delete
3	Bruno	Ferreira	mail@gmail.com	777888	View	Edit	Delete
4	No Accounts	No name	mail@gmail.com	777888	View	Edit	Delete
5	Rui	Ferreira	mail@rui.ferreira.com	888444888	View	Edit	Delete

[New Customer](#)

Form Validation

JSR 303 (part of J2EE) defines a metadata model and API for JavaBeans validation



The screenshot shows a web browser window titled 'JavaBank' with the address bar displaying 'localhost:8090/javabank/customer'. The page content is titled 'Customer Details' and contains four text input fields with associated labels and validation error messages:

- First Name:** The input field contains 'Ru'. Below it, the error message 'size must be between 3 and 64' is displayed.
- Last Name:** The input field is empty. Below it, the error message 'size must be between 3 and 64' is displayed, followed by 'first name is mandatory'.
- Email:** The input field contains 'asdfa'. Below it, the error message 'must be a well-formed email address' is displayed.
- Phone:** The input field is empty. Below it, the error message 'size must be between 9 and 16' is displayed.

A 'Submit' button is located at the bottom left of the form.

Spring MVC is JSR 303 compatible

Hibernate Validator

JSR 303 reference implementation, validation rules expressed in a standardized way using annotation-based constraints

```
<!-- JSR-301/3 validation -->
<dependency>
  <groupId>org.hibernate.validator</groupId>
  <artifactId>hibernate-validator</artifactId>
  <version>6.0.2.Final</version>
</dependency>
```

Form Validation - DTO

Validation rules and error messages are defined through annotations in the DTO class

```
public class CustomerDto {  
  
    private Integer id;  
  
    @NotNull(message = "first name is mandatory")  
    @NotBlank(message = "first name is mandatory")  
    @Size(min=3, max=64)  
    private String firstName;  
  
    @NotNull(message = "last name is mandatory")  
    @NotBlank(message = "last name is mandatory")  
    @Size(min=3, max=64)  
    private String lastName;  
  
    @Email  
    private String email;  
  
    @Pattern(regexp = "^\\+?[0-9]*$", message = "phone has invalid characters")  
    @Size(min=9, max=16)  
    private String phone;  
  
    // Getters and setters ...  
}
```

Form Validation - Controller

@Valid annotation on the command argument

BindingResult object with validation results

```
@Controller
@RequestMapping("customer")
public class CustomerController {

    @RequestMapping(method = RequestMethod.POST, path = {"/", ""})
    public String saveCustomer(@Valid @ModelAttribute CustomerDto newCustomer, BindingResult bindingResult) {

        if (bindingResult.hasErrors()) {
            return "customer/add-update";
        }

        // save customer

    }
}
```

The **BindingResult** argument must immediately follow the command object

Form Validation - View

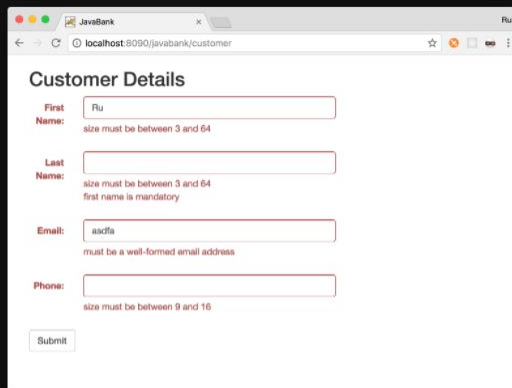
The function **#fields.hasErrors()** can be used to check if a field has errors

With **#fields.errors()** or **th:errors** the errors for a particular field can be displayed

```
<label>First Name:</label>
<input type="text" th:field="*{firstName}"/>
<span th:if="${#fields.hasErrors('firstName')}}" th:errors="*{firstName}">
    First Name field error messages
</span>
```

{ Exercise }

Add form validation to customer add/update form in the JavaBank application



The screenshot shows a web browser window titled "JavaBank" with the URL "localhost:8090/javabank/customer". The page displays a "Customer Details" form with the following fields and validation messages:

- First Name:** Input field contains "Ru". Below the field, the message "size must be between 3 and 64" is displayed.
- Last Name:** Input field is empty. Below the field, the messages "size must be between 3 and 64" and "first name is mandatory" are displayed.
- Email:** Input field contains "asdfs". Below the field, the message "must be a well-formed email address" is displayed.
- Phone:** Input field is empty. Below the field, the message "size must be between 9 and 16" is displayed.

A "Submit" button is located at the bottom left of the form.

Domain Errors



Dealing with the sunshine path in our application is not enough
How should we handle an action on a Customer that does not exist?
Code to handle application domains errors is required!

Domain Exceptions

Creating dedicated exceptions for signaling business logic errors allows specialized error handling for each situation

Lets start small with a single **JavaBankException** and handle all generic business logic errors in one centralized place

How can we achieve this?

AOP to the rescue

Annotating a class with @ControllerAdvice applies its methods to all controllers in the application

Annotations like @ExceptionHandler in a specific method act as selectors, calling the method when the selected exception is thrown

Armed with this information, let's create a generic exception handler

Generic Exception Handler

Controller responsible for rendering
a generic error view on uncaught exceptions

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ResponseStatus(HttpStatus.BAD_REQUEST) // respond with an http 400
    @ExceptionHandler(value = JavaBankException.class) // user did something wrong
    public ModelAndView handleClientErrors(HttpServletRequest req, JavaBankException ex) {

        ModelAndView model = new ModelAndView();

        // add error info to the model
        model.setViewName("error");
        return model;
    }

    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR) // respond with an http 500
    @ExceptionHandler(value = Exception.class) // something is wrong with the server
    public ModelAndView handleServerErrors(HttpServletRequest req, Exception ex) {

        ModelAndView model = new ModelAndView();

        // add error info to the model
        model.setViewName("error");
        return model;
    }
}
```

Application Error

Cause: Customer does not exist

Page: http://localhost:8080/javabank/customer/10

Occurred: Tue Mar 06 15:21:25 WET 2018

Response Status: 400 (Customer does not exist)

Specific Exception Handlers

Now that we have a proper error page for uncaught exceptions, we can add specific error handling logic to controllers

```
@RequestMapping(method = RequestMethod.GET, path =("/{id}/delete")
public String deleteCustomer(@PathVariable Integer id, RedirectAttributes redirectAttributes) {

    try {

        Customer customer = customerService.get(id);
        customerService.delete(id);

        // populate the model and render the appropriate view

    } catch (AssociationExistsException) {

        // provide a visual indication to the user that is not a good idea
        // to delete the customer before closing all accounts

    }

}
```

Concrete business logic exceptions should extend from our generic **JavaBankException**

Logging

What is Logging?



The process of writing messages regarding a running piece of software to a centralized place

These messages have different levels to reflect their importance:

- TRACE
- DEBUG
- INFO
- WARN
- ERROR
- FATAL

The object which performs the logging is called a Logger

Importance and Benefits of Logging

Logging allows us to persist important information such as errors, client accesses, transactions and other relevant events.

Log files are mainly used for debugging, data analysis and statistics.

Stack traces are useful for knowing the exact point where an application failed, but they provide little to no information regarding the previous state of an application

Logging Options



Three major logging frameworks to choose from exist:

- `java.util.logging` (commonly known as JUL)
- Logback
- Log4j/Log4j2

JUL cannot be recommended due to a lower performance when compared to the other options

SLF4J is a logging facade, which abstracts the various logging frameworks from the application



- API Separation
- Support for multiple APIs
- Automatic Reloading of Configurations
- Advanced Filtering
- Plugin Architecture

Log4j2 Dependencies

```
<!-- pom.xml -->
<dependencies>
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.10.0</version>
  </dependency>

  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>2.10.0</version>
  </dependency>

  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-web</artifactId>
    <version>2.10.0</version>
  </dependency>
</dependencies>
```

Creating a Logger with Log4j2

Loggers created with the **LogManager.getLogger()** method

It's accepted as a common practice to request one logger per class.
This way, our log writings are easily identifiable.

```
public class MySpecialClass {  
  
    // getting the class logger  
    // by convention, the property's name is logger  
    private Logger byClass = LogManager.getLogger(MySpecialClass.class);  
    private Logger byName = LogManager.getLogger("org.acemiadecodigo.bootcamp.MySpecialClass");  
  
    public static void main(String[] args) {  
  
        // logging different levels  
        byClass.trace("Trace log message");  
        byClass.debug("Debug log message");  
        byClass.info("Info log message");  
        byClass.warn("Warn log message");  
        byClass.error("Error log message");  
        byClass.fatal("Fatal log message");  
    }  
}
```

Log4j2 Configuration

Log4j2 can be configured under various formats:

- Properties File
- YAML File
- JSON
- XML File

Its two main elements are **Appenders** and **Loggers**

Appenders

Appenders write event data to a target location. We can choose to write this data under a given format by using **layouts**.

A layout allows us to choose which data elements we want to include on each of the records written by a specific appender

Appenders

```
Logger logger = LogManager.getLogger(MySpecialClass.class);  
logger.info("Logging is the activity of cutting trees");
```



Appender

Layout

`(%d - %p - %c - %m%n)`

date - priority/level - class - message



2018-02-12 17:36:49,876 - INFO - org.academiadecodigo.bootcamp.controller.MySpecialClass - Logging is the activity of cutting trees

(%n is simply the new line separator)

Loggers

Objects which we have to interact with in order to perform logging.

Each logger can have multiple appenders and will delegate to one of the appenders the operation of writing to an output.

At the very least the Root logger must be configured!

Javabank Log4j2 XML Configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="ALL">
  <Appenders>
    <RollingFile name="AccessRollingFile"
      fileName="${sys:catalina.home}/logs/javabank/access/javabank_access.log"
      filePattern="${sys:catalina.home}/logs/javabank/access/${date:yyyy}/${date:MM}/javabank_access_%d{dd}.log"
      append="false">
      <PatternLayout>
        <Pattern>%d [%t] - %p - %c - %m%n</Pattern>
      </PatternLayout>
      <Policies>
        <TimeBasedTriggeringPolicy interval="1" modulate="true"/>
      </Policies>
      <LevelRangeFilter minLevel="INFO" maxLevel="INFO"/>
    </RollingFile>

    <RollingFile name="ErrorRollingFile"
      fileName="${sys:catalina.home}/logs/javabank/error/javabank_error.log"
      filePattern="${sys:catalina.home}/logs/javabank/error/${date:yyyy}/${date:MM}/javabank_error_%d{dd}.log"
      append="false">
      <PatternLayout>
        <Pattern>%d [%t] - %p - %c - %m%n</Pattern>
      </PatternLayout>
      <Policies>
        <TimeBasedTriggeringPolicy interval="1" modulate="true"/>
      </Policies>
      <ThresholdFilter level="WARN" onMatch="ACCEPT" onMismatch="DENY"/>
    </RollingFile>

    <RollingFile name="DebugRollingFile" fileName="${sys:catalina.home}/logs/javabank/debug/javabank_debug.log"
      filePattern="${sys:catalina.home}/logs/javabank/debug/${date:yyyy}/${date:MM}/javabank_debug_%d{dd}.log"
```

Javabank Log4j2 XML Configuration

```
</Pattern>%d [%t] - %p - %c - %n%</Pattern>
</PatternLayout>
<Policies>
  <TimeBasedTriggeringPolicy interval="1" modulate="true"/>
</Policies>
<ThresholdFilter level="WARN" onMatch="ACCEPT" onMismatch="DENY"/>
</RollingFile>

<RollingFile name="DebugRollingFile" fileName="${sys:catalina.home}/logs/javabank/debug/javabank_debug.log"
  filePattern="${sys:catalina.home}/logs/javabank/debug/${date:yyyy}/${date:MM}/javabank_debug_%d{dd}.log"
  append="false">
  <PatternLayout>
    <Pattern>%d [%t] - %p - %c - %n%</Pattern>
  </PatternLayout>
  <Policies>
    <TimeBasedTriggeringPolicy interval="1" modulate="true"/>
  </Policies>
</RollingFile>

</Appenders>
<Loggers>
  <Root level="INFO">
    <AppenderRef ref="AccessRollingFile"/>
    <AppenderRef ref="ErrorRollingFile"/>
  </Root>

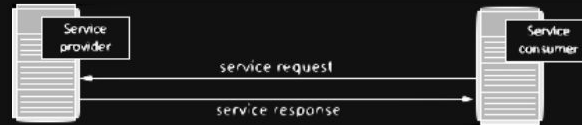
  <Logger name="org.hibernate.SQL" level="DEBUG" additivity="false">
    <AppenderRef ref="DebugRollingFile"/>
  </Logger>
</Loggers>
</Configuration>
```

Web Services



Service Oriented Architecture (SOA)

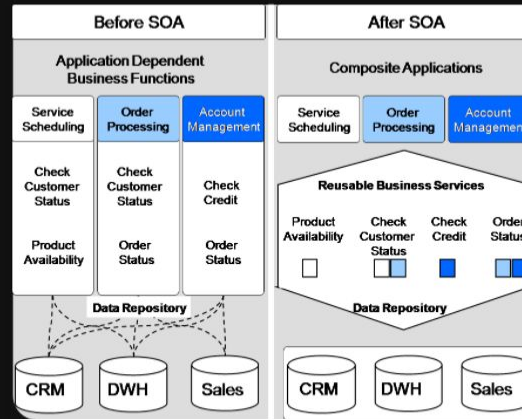
- Models an application as a composition of reusable services
- These services are exposed on the network and reusable by multiple applications



Popular Distributed Application Frameworks
used to implement SOA include:

- COM, DCOM - Microsoft Specific
- EJB, RMI - Java Specific
- CORBA, XML-RPC, SOAP - Platform and language independent

Benefits of SOA



- Reusability of new and existing software systems
- Loose Coupling of software components
- Flexibility to use a variety of platforms and languages
- Easy to Scale individual services without affecting others
- Services can be tested and debugged in isolation

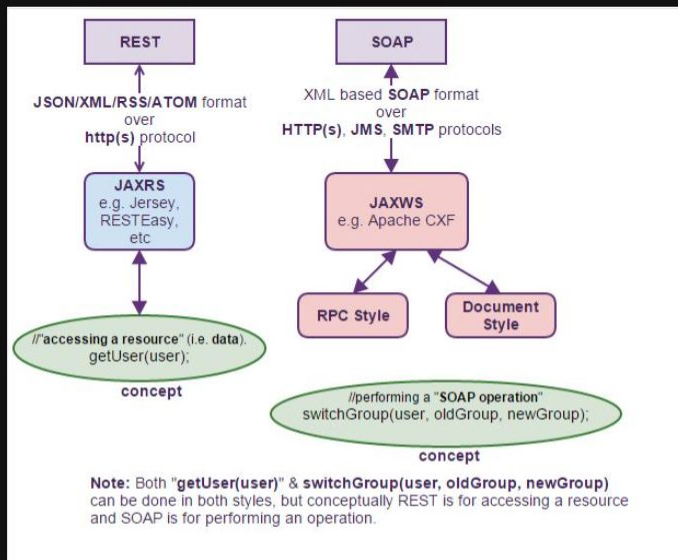
What is a Web Service?

Usage of the Web Platform as a connection technology for service-oriented architectures. The benefits are:

- Exposing of existing function on the network, allowing other applications to remotely invoke our code
- Interoperability, allowing various applications to talk to each other and share data and services among themselves, independently of their platforms/technologies
- Standardized Protocols, by usage of standardized industry protocols for the communication
- Low Cost of Communication as existing low-cost internet can be used for implementing web services rather than proprietary B2B solutions

Types of WebServices

- SOAP - exposes business logic operations
- RESTful - exposes CRUD operations on data

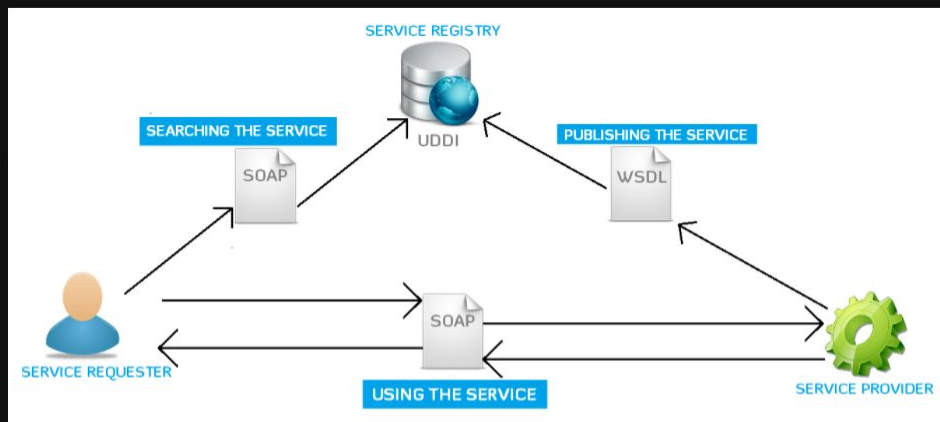


SOAP - Simple Object Access Protocol

Contract First Web Services



- Powerful
- Versatile
- Automated Tools
- WSDL Contract
- XML Payloads
- Too Verbose..

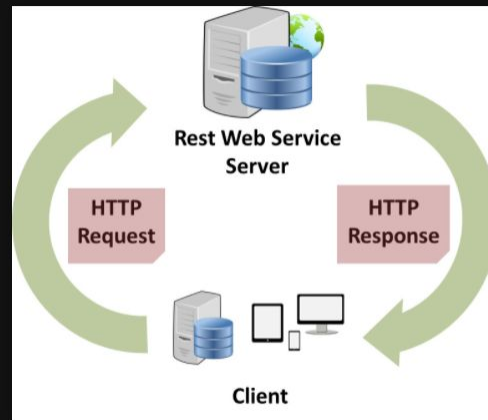


REST - Representational State Transfer

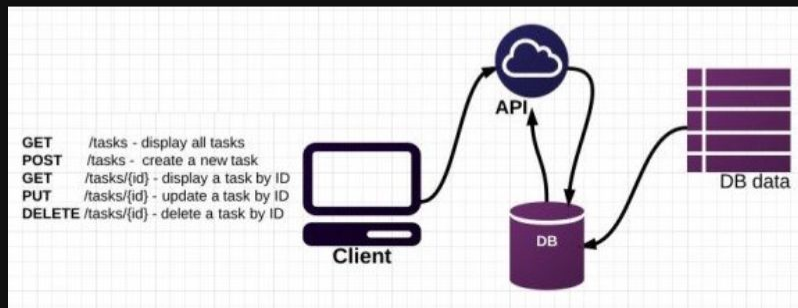


Architectural style in which systems communicate over HTTP using the same verbs that web browsers use to retrieve web pages and to send data to remote servers. Such systems are called RESTful.

- Not a protocol
- Uses HTTP
- Stateless
- Cacheable
- Exposes CRUD
- Legacy XML
- Modern JSON



REST API Design

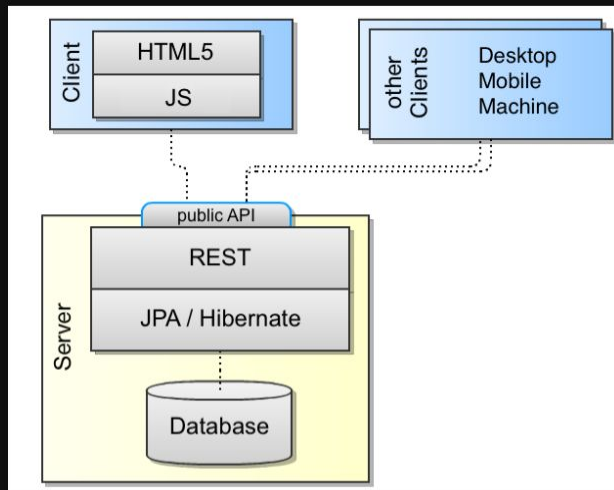


HTTP Verb	/tasks	/tasks/{id}
GET	200 (OK), list of tasks. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single task. 404 (Not Found), if ID not found or invalid.
PUT	404 (Not Found), unless you want to update/replace every resource in the entire collection.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
POST	201 (Created). Response also contains a link to the created task (/task/{id}) in its header.	404 (Not Found).
DELETE	404 (Not Found), unless you want to delete the whole collection which is not often desirable.	200 (OK). 404 (Not Found), if ID not found or invalid.

Modern Application Design

Services are exposed as a REST API

API can be consumed by different types of clients/devices



JSON vs XML



- Native to JavaScript
- Simplicity
- No Defined Data Structures
- No Well Defined Standards
- Higher Performance
- Validation
- Consistency
- Strongly Typed
- Strict Structure
- Lower Performance

```
http://localhost:8080/Json/SyncReply/Contacts
{
  - Contacts: [
    - {
      FirstName: "Demis",
      LastName: "Bellot",
      Email: "demis.bellot@gmail.com"
    },
    - {
      FirstName: "Steve",
      LastName: "Jobs",
      Email: "steve@apple.com"
    },
    - {
      FirstName: "Steve",
      LastName: "Ballmer",
      Email: "steve@microsoft.com"
    },
    - {
      FirstName: "Eric",
      LastName: "Schmidt",
      Email: "eric@google.com"
    },
    - {
      FirstName: "Larry",
      LastName: "Ellison",
      Email: "larry@oracle.com"
    }
  ]
}

http://localhost:8080/XML/SyncReply/Contacts
<ContactsResponse xmlns:i="http://www.w3.org/2003/05/soap-envelope">
  <Contacts>
    <Contact>
      <Email>demis.bellot@gmail.com</Email>
      <FirstName>Demis</FirstName>
      <LastName>Bellot</LastName>
    </Contact>
    <Contact>
      <Email>steve@apple.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Jobs</LastName>
    </Contact>
    <Contact>
      <Email>steve@microsoft.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Ballmer</LastName>
    </Contact>
    <Contact>
      <Email>eric@google.com</Email>
      <FirstName>Eric</FirstName>
      <LastName>Schmidt</LastName>
    </Contact>
    <Contact>
      <Email>larry@oracle.com</Email>
      <FirstName>Larry</FirstName>
      <LastName>Ellison</LastName>
    </Contact>
  </Contacts>
</ContactsResponse>
```

Jackson

Standard JSON library for Java

Provides a framework to serialize custom java objects to a json string and deserialize a json string back into a java objects.

```
<dependency>  
  <groupId>com.fasterxml.jackson.core</groupId>  
  <artifactId>jackson-databind</artifactId>  
  <version>2.8.0</version>  
</dependency>
```

How do we use it?

```
import com.fasterxml.jackson.databind.ObjectMapper;  
  
// ...  
  
ObjectMapper mapper = new ObjectMapper();
```

JSON to POJO

Object Deserialization

```
public class MyPojo {  
  
    private String name;  
    private int age;  
  
    // Getters and Setters  
  
}  
  
MyPojo myPojo1 = mapper.readValue(new File("data.json"), MyPojo.class);  
MyPojo myPojo2 = mapper.readValue(new URL("http://api.com/data.json"), MyPojo.class);  
MyPojo myPojo3 = mapper.readValue("{\"name\":\"Pedro\", \"age\":23}", MyPojo.class);
```

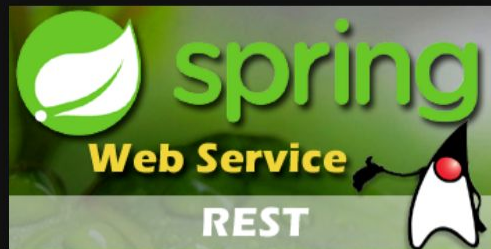
POJO to JSON

Object Serialization

```
public class MyPojo {  
  
    private String name;  
    private int age;  
  
    // Getters and Setters  
  
}  
  
MyPojo myPojo = new MyPojo();  
myPojo.setName("Pedro");  
myPojo.setAge(23);  
  
mapper.writeValue(new File("result.json"), myPojo);  
byte[] jsonBytes = mapper.writeValueAsBytes(myPojo);  
String jsonString = mapper.writeValueAsString(myPojo);
```

{ Exercise }

Fetch IP information from ipinfo.io public JSON API



Spring REST API

@RequestBody and @ResponseBody can be used in a controller to implement smart object serialization and deserialization

They extract the logic of message conversion from the code allow multiple data formats for a single resource without code duplication

Rest Controller

- If a method parameter is annotated with `@RequestBody`, spring will try to convert the content of the incoming request body to a parameter object on the fly
- If a method is annotated with `@ResponseBody`, spring will try to convert its return value and write it to the http response automatically

```
@Controller
@ResponseBody
public class HelloWorldController {

    @RequestMapping(method = RequestMethod.GET, value = "api/hello")
    public String helloWorld(@RequestParam(value = "name", defaultValue = "World") String str) {

        return "Hello " + str;
    }
}
```

Annotating a class with `@RestController` is the same as annotating it with both `@Controller` and `@ResponseBody`.

Response Entity

The ResponseEntity class represents an HTTP response, consisting of headers, body and response code

It is used to return a specific http status code from the controller

```
@RestController
public class HelloWorldController {

    @RequestMapping(method = RequestMethod.GET, value = "api/hello")
    public ResponseEntity<String>
        helloWorld(@RequestParam(value = "name", defaultValue = "World") String str) {

        return new ResponseEntity<String>("Hello " + str, HttpStatus.I_AM_A_TEAPOT);

    }
}
```

{ Exercise }

Hello REST API

JSON API

Spring will use Jackson to serialize and deserialize objects automatically if it finds it in the class path

```
public class Message {  
    private String message;  
  
    // getters and setters  
}  
  
@RestController  
public class HelloWorldController {  
    @RequestMapping(  
        method = RequestMethod.GET,  
        value = "api/hello",  
        produces = MediaType.APPLICATION_JSON_VALUE  
    )  
    public ResponseEntity<Message>  
        helloWorld(@RequestParam(value = "name", defaultValue = "World") String str) {  
  
        Message message = new Message();  
        message.setMessage("Hello " + str);  
        return new ResponseEntity<>(message, HttpStatus.OK);  
    }  
}
```

JSON Validation

Remember the `@Valid` annotation? We can validate the deserialized JSON object by adding `@Valid` to arguments

By using the `BindingResult` class we can access the evaluation result

```
@RestController
public class UserController {

    @RequestMapping(
        method = RequestMethod.POST,
        value = "api/users",
        produces = MediaType.APPLICATION_JSON_VALUE
    )
    public ResponseEntity addUser(@Valid @RequestBody User user, BindingResult bindingResult) {

        if (bindingResult.hasErrors()) {
            return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
        }

        // ...
    }
}
```

The **BindingResult** argument must immediately follow the object to be validated

{ Exercise }

Add a JSON REST API with Customer CRUD operations to the JavaBank application

Test with Postman / curl

Http Method	EndPoint	Action
GET	/customer	read all customers, no account info
GET	/customer/{id}	read a specific customer
GET	/customer/{id}/account	read all accounts for customer
GET	/customer/{cid}/account/{aid}	read account for customer
POST	/customer	create a new customer
PUT	/customer/{id}	edit an existing customer
DELETE	/customer/{id}	delete an existing customer
DELETE	/customer/{cid}/account/{aid}	delete an account

Web Application Security



One of the most misunderstood and misimplemented concepts

Security is like onions, it comes in layers

TCP/IP stack security

Traditional web applications communicate through **HTTP**,
which by nature is insecure

A secure approach to communication between client and server is
using **HTTPS (HTTP + TLS)**

This can be provided by your application container e.g: **Tomcat**

Here's a quick how to

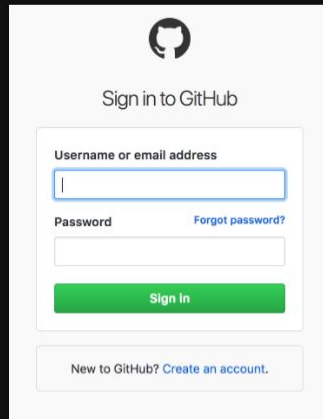
Enterprise Application Security

This an extensive topic and we'll only cover mostly 2 topics:

- **Authentication** - Determines if the user is who he says he is
- **Authorization** - Determines if the user has adequate permissions to perform some action or access some resource

Authentication

The most common authentication mechanism is the client providing the server with a combination of a user and password



The image shows a screenshot of the GitHub sign-in interface. At the top is the GitHub logo (Octocat). Below it is the text "Sign in to GitHub". The form contains two input fields: "Username or email address" and "Password". The "Username or email address" field has a blue border and a cursor. The "Password" field is a standard white box. To the right of the password field is a blue link that says "Forgot password?". Below the password field is a green "Sign In" button. At the bottom of the form is a link that says "New to GitHub? Create an account."

Examples of other mechanisms of authentication are: PIN and Hardware

Authentication

We'll be rolling our authentication mechanisms
with the help of the Spring Framework

- A combination of username and password for each customer has to be stored in the database
- Before the customer can interact with the application, a valid set of credentials has to be provided

Is storing that information in clear text secure enough?
Probably not, we'll get back to this in a jiffy.

Authorization

Right now, every single user of our application can perform any action, such as transfer funds from an account that isn't theirs

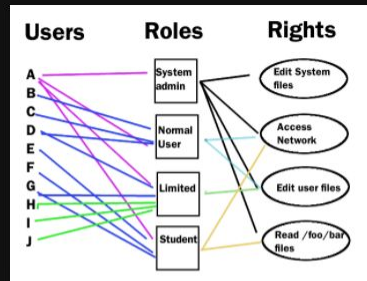
Of course, we don't want that to happen!

Spring will also help us to configure that

Authorization

Who should be able to access a specific operation in our application?

To better answer this question, instead of saying user x, y and z we can provide each one of them a role



Each user can accumulate several roles within an application

Spring Security

De facto security framework for JVM based languages

Reference guide



Dependencies

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-web</artifactId>
  <version>4.2.4.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-config</artifactId>
  <version>4.2.4.RELEASE</version>
  <scope>runtime</scope>
</dependency>
```

Bootstrapping Spring Security

```
<!-- All requests will pass through the Spring Security filter -->
<filter>
  <filter-name>springSecurityFilterChain</filter-name>
  <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>

<filter-mapping>
  <filter-name>springSecurityFilterChain</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

Spring Security configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:security="http://www.springframework.org/schema/security"
       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/security http://www.springframework.org/schema/security"
       >

    <security:http>
        <security:form-login/>
        <security:intercept-url pattern="/**" access="hasAnyRole('ADMIN', 'USER')"/>
    </security:http>

    <security:authentication-manager>
        <security:authentication-provider>
            <security:user-service>
                <security:user name="admin" password="password" authorities="ROLE_ADMIN"/>
                <security:user name="user" password="password" authorities="ROLE_USER"/>
            </security:user-service>
        </security:authentication-provider>
    </security:authentication-manager>
</beans>
```

{ Hands On }

Integrate Spring Security into JavaBank

Spring Security



Has a lot of built-in behaviour but this setup is rarely used
in a production ready application

To better match our needs, it is possible to override the default
behaviour

Overriding Spring Security defaults

Application users are declared in configuration
Good for testing purposes, bad for production

**Create User and Role models and
override the default Spring Security behaviour**

User Model

```
@Entity
@Table(name = "user")
public class User extends AbstractModel {

    private String name;
    private String password;

    @OneToOne
    private Customer customer;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "USER_ROLE",
        joinColumns = @JoinColumn(name = "USER_ID", referencedColumnName = "ID"),
        inverseJoinColumns = @JoinColumn(name = "ROLE_ID", referencedColumnName = "ID")
    )
    private Set<Role> roles;

    // getters and setters

}
```

Role Model

```
@Entity
@Table(name = "role")
public class Role extends AbstractModel {

    private String name;

    // getters and setters

}
```

User Service

We need to provide Spring our own UserService implementation

```
<security:authentication-manager>  
  <security:authentication-provider user-service-ref="userServiceImpl" />  
</security:authentication-manager>
```

The user service will be responsible for
retrieving the user data from the database

UserDetailsService

UserService has to implement the **UserDetailsService** interface

loadUserByUsername() locates a user by its username

```
@Component
public class UserServiceImpl implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        return new User("admin", "password", Arrays.asList(
            new SimpleGrantedAuthority("ROLE_ADMIN"),
            new SimpleGrantedAuthority("ROLE_USER")
        ));
    }
}
```

Refer to [UserDetailsService](#) and [Spring Security User](#) API docs

{ Exercise }

Integrate the User and Role entities with Spring Security

SELECT * FROM user;

Right now, your user's password is stored in clear text,
which is NOT secure at all!

Sensitive information should always be hashed before stored

Let's implement our own hashing algorithm! Not really, although this
would have been fun. Don't roll your own cryptography!

bcrypt

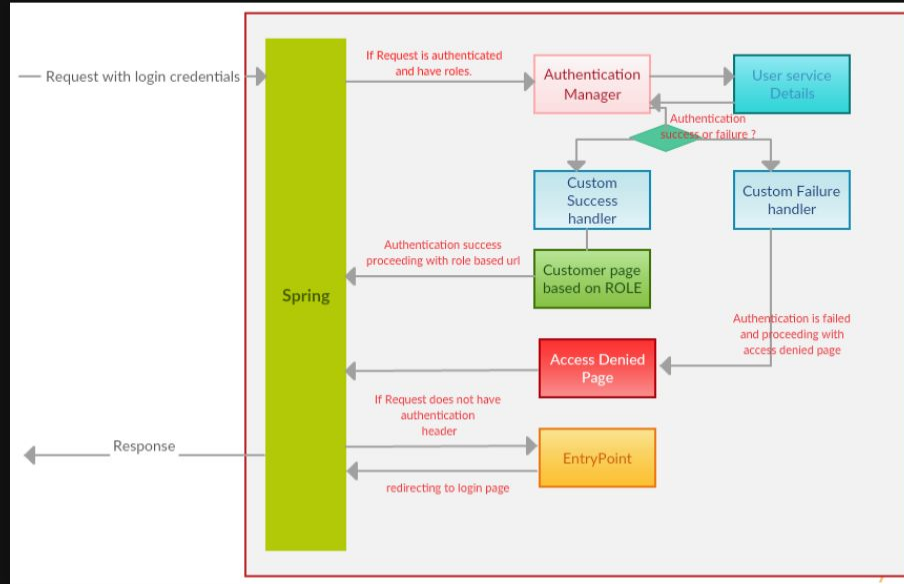
bcrypt is one of the most famous hashing algorithms and a good choice for encrypting sensitive information

```
<security:authentication-manager>
  <security:authentication-provider user-service-ref="userServiceImpl">
    <security:password-encoder ref="bcryptEncoder"/>
  </security:authentication-provider>
</security:authentication-manager>

<bean
  name="bcryptEncoder"
  class="org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder"
/>
```

Voilà!

Request flow chart



Role based Redirect

A common application requirement is to redirect different types of users to different pages after login

We can achieve this by configuring Spring to use our Custom Authentication Success Handler

```
<security:http>
  <security:form-login authentication-success-handler-ref="myCustomAuthenticationSuccessHandl
  <security:intercept-url pattern="/customer/list" access="hasRole('ADMIN')"/>
</security:http>
```

We can implement our custom authentication success handler by implementing **SimpleUrlAuthenticationSuccessHandler**

Expression-Based Access Control

Complex rules can be transformed into a simple boolean expression using expression based access control

```
<security:http>  
  <security:intercept-url pattern="/customer/{customerId}/**"  
    access="hasRole('ADMIN') or @permissionEvaluator.hasPermission(authentication, #customerId)"/>  
</security:http>
```

This expression prevents customers to access other customer pages, unless they have the **ADMIN** role

{ Exercise }

Implement role based access control in the JavaBank application using Spring Security